INTERACAO 2

Sistema de Gerenciamento de Tarefas Pessoais

Inicio da Implementacao | Requisitos Ajustados | Arquitetura C4

Disciplina: Programacao WEB
Data: 16/03/2026

1. Resposta a Interacao 2

1.1 Decisao de Inicio: Front-end (Thymeleaf)

A implementacao do projeto foi iniciada pelo Front-end, por meio da criacao da estrutura inicial de templates Thymeleaf (scaffolding). Esta decisao foi baseada nos seguintes criterios:

- Validacao visual antecipada: construir os templates HTML primeiro permite validar o fluxo de navegacao e a experiencia do usuario antes de a logica de negocio estar completa.
- Compatibilidade com MVC: no Spring MVC, as views (Thymeleaf) sao independentes do back-end e podem ser desenvolvidas com dados mockados antes da persistencia real.
- Reducao de retrabalho: definir formularios e campos primeiro evita que o back-end seja implementado com estruturas que nao correspondem ao que a interface necessita.
- Simplicidade do escopo: projeto individual de pequeno porte favorece ciclos de feedback rapidos com entregas incrementais claras.

1.2 Artefatos no Repositorio

Os seguintes artefatos foram commitados no repositorio do projeto como resultado desta interacao:

Arquivo	Descricao
src/.../templates/lista.html	Template principal. Lista tarefas com filtro por status e botoes de acao.
src/.../templates/nova.html	Formulario de cadastro (titulo, descricao, data de vencimento).
src/.../templates/editar.html	Formulario de edicao com pre-carregamento de dados.
src/.../templates/fragments/	Fragmentos reutilizaveis: header.html e footer.html.
src/.../static/css/style.css	Folha de estilo customizada com variaveis CSS e classes de status.
src/.../model/Tarefa.java	Entidade JPA com todos os atributos e @PrePersist definidos.
src/.../model/StatusTarefa.java	Enum: PENDENTE | CONCLUIDA.
src/.../TarefasApplication.java	Classe principal Spring Boot — ponto de entrada.
pom.xml	Dependencias Maven: Spring Boot, Thymeleaf, H2, JPA, Validation, Bootstrap CDN.
README.md	Descricao do projeto, instrucoes de execucao e estrutura de pastas.

2. Estrutura Inicial (Scaffolding)

2.1 Estrutura de Pastas

```
tarefas/
├── pom.xml
├── README.md
└── src/main/
    ├── java/com/example/tarefas/
    │   ├── TarefasApplication.java
    │   ├── model/
    │   │   ├── Tarefa.java
    │   │   └── StatusTarefa.java
    │   ├── repository/
    │   │   └── TarefaRepository.java   (stub)
    │   ├── service/
    │   │   └── TarefaService.java       (stub)
    │   └── controller/
    │       └── TarefaController.java    (stub)
    └── resources/
        ├── application.properties
        ├── static/css/style.css
        └── templates/
            ├── fragments/
            │   ├── header.html
            │   └── footer.html
            ├── lista.html
            ├── nova.html
            └── editar.html
```

2.2 pom.xml — Dependencias Maven

```
<dependencies>
  <!-- Web + Thymeleaf -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
  <!-- Persistencia -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>
  <!-- Validacao -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
```

```
  </dependency>
</dependencies>
```

2.3 Tarefa.java — Entidade JPA

```
@Entity
public class Tarefa {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "O titulo e obrigatorio")
    private String titulo;

    private String descricao;
    private LocalDate dataCriacao;
    private LocalDate dataVencimento;

    @Enumerated(EnumType.STRING)
    private StatusTarefa status;

    @PrePersist
    public void prePersist() {
        this.dataCriacao = LocalDate.now();
        if (this.status == null) {
            this.status = StatusTarefa.PENDENTE;
        }
    }
    // getters e setters
}
```

2.4 lista.html — Template Principal

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3/dist/css/bootstrap.min.css"/>
  <link rel="stylesheet" th:href="@{/css/style.css}"/>
</head>
<body>
  <div class="container mt-4">
    <div class="d-flex justify-content-between mb-3">
      <h1 class="h3">Minhas Tarefas</h1>
      <a th:href="@{/tarefas/nova}" class="btn btn-primary">+ Nova Tarefa</a>
    </div>
    <!-- Filtros -->
    <div class="btn-group mb-3">
      <a th:href="@{/tarefas}"                      class="btn btn-outline-
secondary">Todas</a>
      <a th:href="@{/tarefas(status='PENDENTE')}"   class="btn btn-outline-warning"
>Pendentes</a>
      <a th:href="@{/tarefas(status='CONCLUIDA')}"  class="btn btn-outline-success"
>Concluidas</a>
    </div>
```

```
    <!-- Flash message -->
    <div th:if="${msg}" class="alert alert-success" th:text="${msg}"></div>
    <!-- Lista -->
    <div th:each="t : ${tarefas}" class="card mb-2">
      <div class="card-body d-flex justify-content-between align-items-center">
        <div>
          <h5 th:text="${t.titulo}" class="mb-1"></h5>
          <small th:text="${t.descricao}" class="text-muted"></small>
          <span th:text="${t.status}"
                th:class="${t.status.name()=='PENDENTE'} ? 'badge bg-warning text-
dark'
                                                          : 'badge bg-
success'"></span>
        </div>
        <div class="btn-group btn-group-sm">
          <a th:href="@{/tarefas/{id}/status(id=${t.id})}" class="btn btn-outline-
secondary">
            <span th:text="${t.status.name()=='PENDENTE'} ? 'Concluir' :
'Reabrir'"></span>
          </a>
          <a th:href="@{/tarefas/{id}/editar(id=${t.id})}"  class="btn btn-outline-
primary">Editar</a>
          <a th:href="@{/tarefas/{id}/excluir(id=${t.id})}"
             onclick="return confirm('Excluir esta tarefa?')"
             class="btn btn-outline-danger">Excluir</a>
        </div>
      </div>
    </div>
    <p th:if="${#lists.isEmpty(tarefas)}" class="text-muted">Nenhuma tarefa
encontrada.</p>
  </div>
</body>
</html>
```

2.5 application.properties

```
spring.datasource.url=jdbc:h2:mem:tarefasdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.thymeleaf.cache=false
```

3. Requisitos Ajustados

Durante a fase de scaffolding foram identificados ajustes pontuais. Nenhum requisito foi removido — apenas refinamentos de escopo, detalhamentos e uma nova funcionalidade (RF07) e um novo requisito nao funcional (RNF08) identificados como necessarios.

3.1 Requisitos Funcionais

ID	Nome	Descricao Ajustada	Status do Ajuste
RF01	**Cadastrar Tarefa**	titulo (@NotBlank), descricao e dataVencimento (opcionais). dataCriacao e status=PENDENTE via @PrePersist.	Sem alteracao
RF02	**Listar Tarefas**	Exibir titulo, badge de status colorido, dataCriacao e dataVencimento formatada (dd/MM/yyyy). Mensagem quando lista vazia.	Ajustado: badge e formato de data
RF03	**Editar Tarefa**	Editar titulo, descricao e dataVencimento. O campo dataCriacao e bloqueado para edicao pelo usuario.	Ajustado: dataCriacao bloqueada
RF04	**Excluir Tarefa**	Excluir permanentemente. Adicionar dialogo de confirmacao JavaScript (confirm()) para evitar exclusoes acidentais.	Ajustado: confirmacao JS
RF05	**Alternar Status**	Botao exibe proximo estado dinamicamente: 'Concluir' quando PENDENTE; 'Reabrir' quando CONCLUIDA.	Ajustado: label dinamico
RF06	**Filtrar Tarefas**	Filtrar por ?status=PENDENTE ou ?status=CONCLUIDA na URL. Botao do filtro ativo com destaque visual (Bootstrap active).	Ajustado: botao ativo
RF07	**Feedback Visual**	NOVO: Flash message de sucesso apos criar, editar, excluir e alternar status. Implementado via RedirectAttributes.	**NOVO requisito**

3.2 Requisitos Nao Funcionais

ID	Categoria	Descricao	Status
RNF01	Back-end Java	Back-end em Java com Spring Boot (DI + IoC).	Mantido
RNF02	Padrao MVC	Arquitetura MVC obrigatoria dentro do monolito.	Mantido
RNF03	Thymeleaf	Front-end renderizado pelo servidor com Thymeleaf.	Mantido
RNF04	H2 + JPA	Spring Data JPA + banco H2 em memoria.	Mantido
RNF05	Maven	Gerenciamento de dependencias com Apache Maven.	Mantido
RNF06	Validacao	Campo titulo @NotBlank no back-end + atributo required no HTML5.	Ajustado

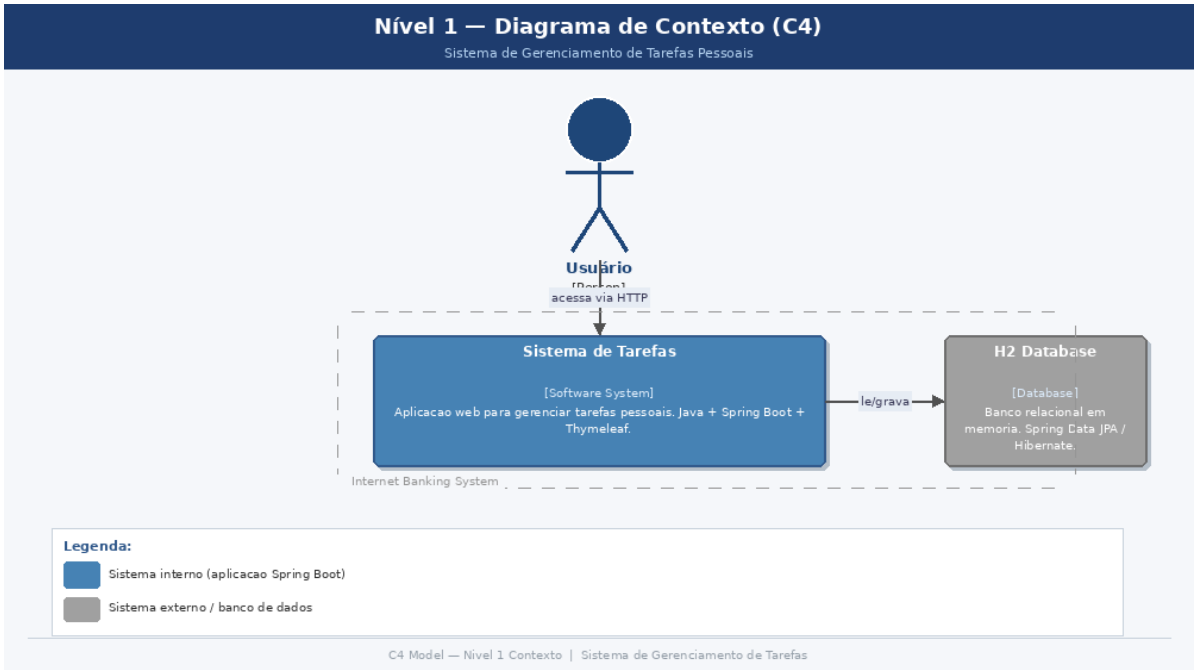
RNF07	Responsividade	Interface utilizavel em desktop >= 1024px de largura.	Mantido
RNF08	Bootstrap 5	NOVO: Adotado Bootstrap 5 via CDN para agilizar componentes visuais (grid, botoes, badges, cards) sem JS externo.	NOVO

4. Arquitetura em Modelo C4

A arquitetura do sistema e descrita por meio do modelo C4 (Context, Containers, Components, Code), organizado em quatro niveis de abstracao progressivamente mais detalhados, conforme o exercicio apresentado na disciplina de Projeto e Arquitetura de Software.

4.1 Nivel 1 — Diagrama de Contexto

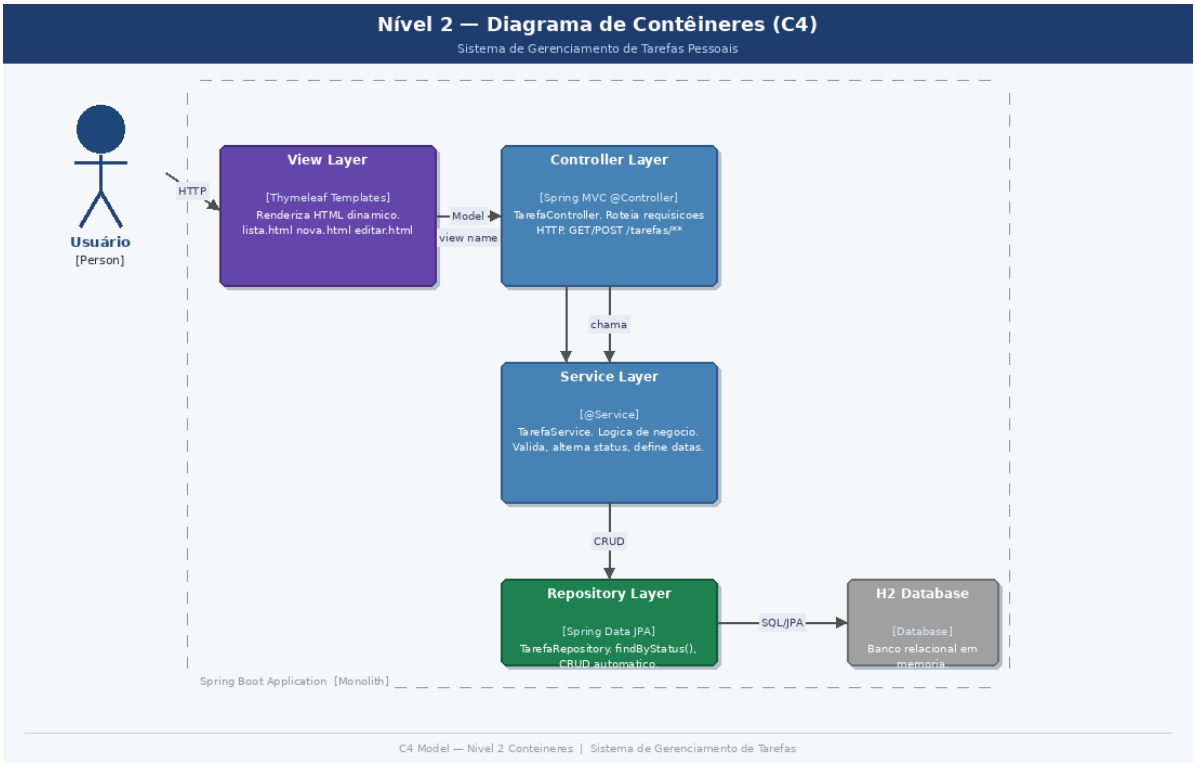
Apresenta o sistema como caixa preta, mostrando o usuario e o banco externo com os quais ele se comunica.



Elemento	Descricao
Usuario	Pessoa fisica que acessa a aplicacao via browser para gerenciar tarefas pessoais.
Sistema de Tarefas	Aplicacao web monolitica (Java + Spring Boot + Thymeleaf). Logica e interface centralizadas.
H2 Database	Banco relacional em memoria. Persiste entidades Tarefa via Spring Data JPA / Hibernate.

4.2 Nivel 2 — Diagrama de Conteineres

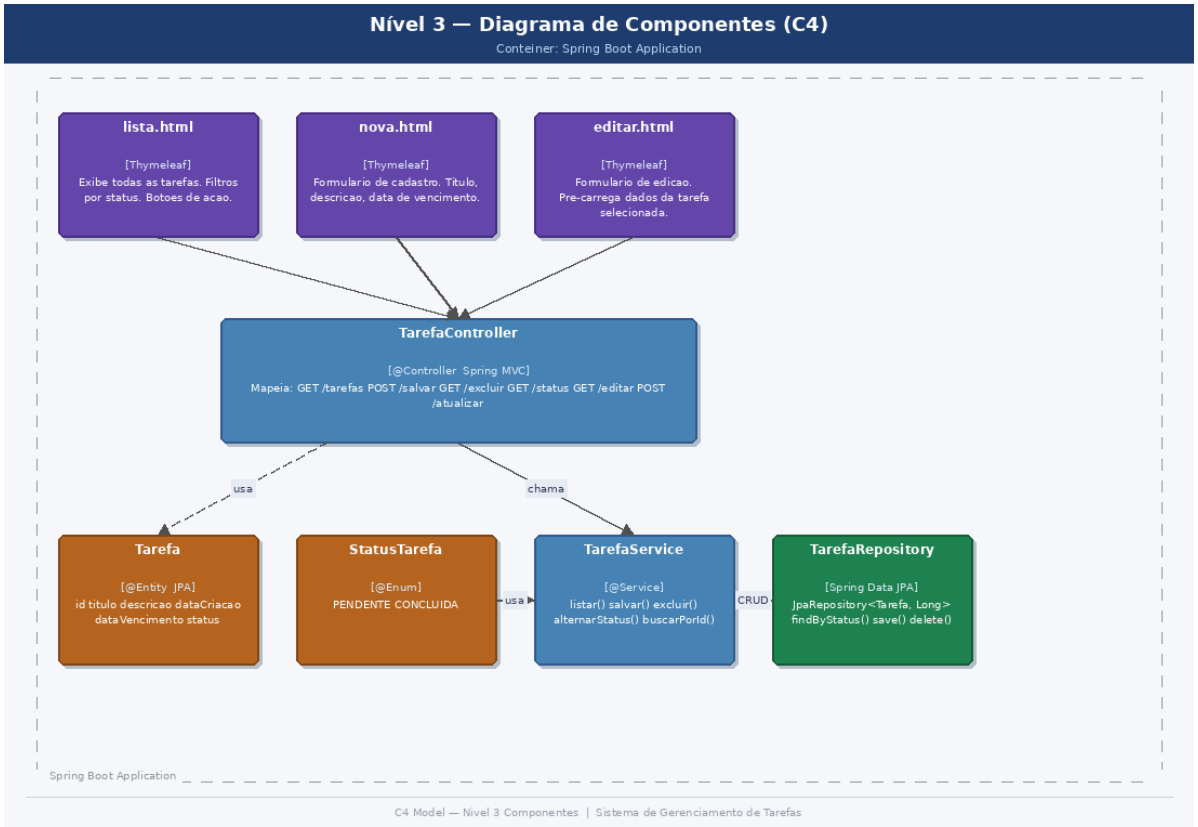
Detalha as unidades logicas de execucao: View, Controller, Service, Repository (todos num unico processo Spring Boot) e o banco H2 externo.



Conteiner	Descricao
View Layer	Templates Thymeleaf. Renderiza HTML dinamico no servidor.
Controller Layer	TarefaController (@Controller). Recebe HTTP e aciona o Service.
Service Layer	TarefaService (@Service). Logica de negocio e regras de validacao.
Repository Layer	TarefaRepository (JPA). CRUD automatico + findByStatus().
H2 Database	Banco relacional externo ao processo Spring Boot.

4.3 Nivel 3 — Diagrama de Componentes

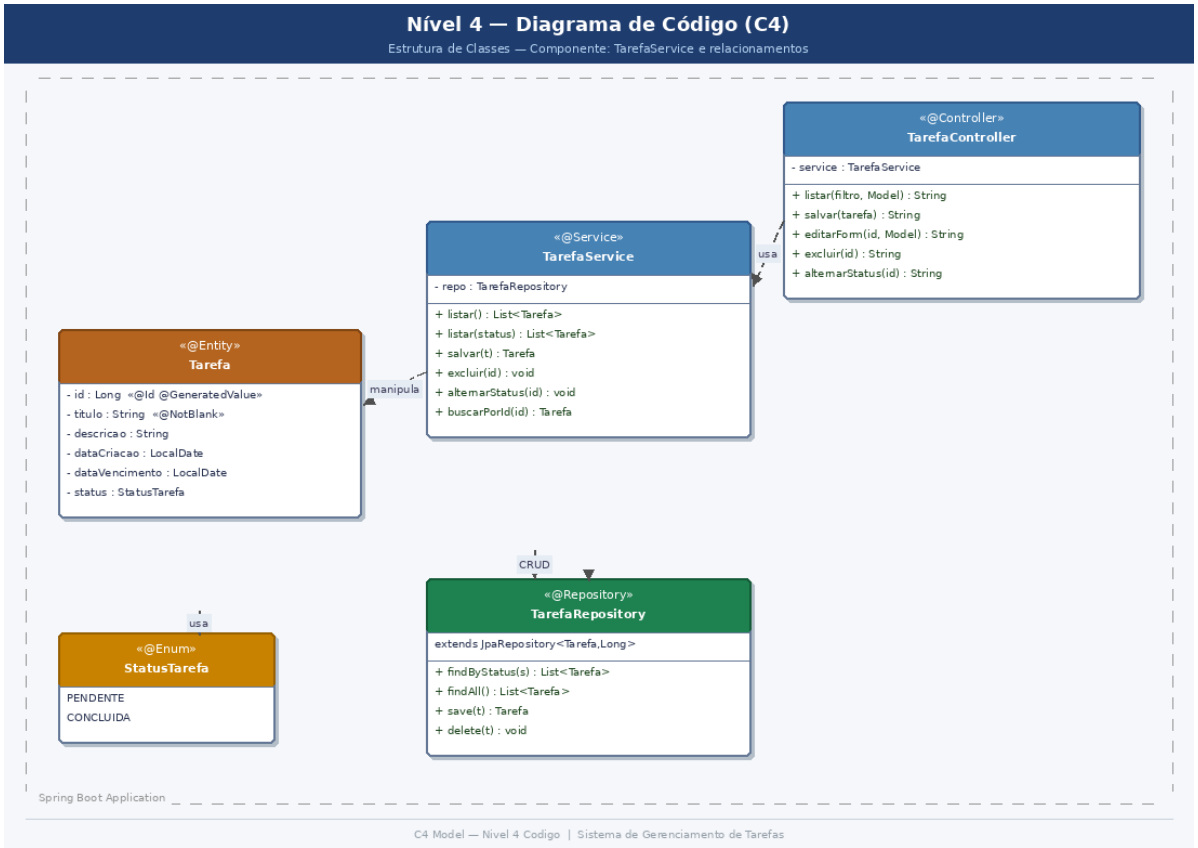
Mapeia os componentes internos do conteiner Spring Boot Application, responsabilidades e dependencias.



Componente	Descricao
lista.html	Listagem de tarefas com filtros por status e botoes de acao.
nova.html	Formulario de cadastro (titulo*, descricao, dataVencimento).
editar.html	Formulario de edicao com pre-carregamento dos dados.
TarefaController	Mapeia as 7 rotas do sistema (GET e POST).
Tarefa	@Entity JPA: id, titulo, descricao, dataCriacao, dataVencimento, status.
StatusTarefa	@Enum: PENDENTE | CONCLUIDA.
TarefaService	listar, salvar, excluir, alternarStatus, buscarPorId.
TarefaRepository	JpaRepository<Tarefa,Long> com findByStatus() customizado.

4.4 Nivel 4 — Diagrama de Codigo

Estrutura de classes UML com atributos, metodos, anotacoes Spring e relacoes de dependencia entre as classes principais.



Classe	Descricao
TarefaController	Injeta TarefaService. Expoe: listar(), salvar(), editarForm(), excluir(), alternarStatus().
TarefaService	Injeta TarefaRepository. Implementa: listar(), salvar(), excluir(), alternarStatus(), buscarPorId().
TarefaRepository	extends JpaRepository<Tarefa,Long>. Adiciona findByStatus(StatusTarefa).
Tarefa	@Entity: id(PK), titulo(@NotBlank), descricao, dataCriacao, dataVencimento, status.
StatusTarefa	@Enum: PENDENTE | CONCLUIDA. Referenciado em Tarefa e nos filtros do Service.